COURSE CODE	COURSE TITLE	L	T	P	C
	Full Stack Application Development	**1**	**0**	**4**	**3**

A. Preamble

This course provides practical knowledge of database systems, web development, backend programming, and cloud technologies. Students gain hands-on experience in building dynamic web applications using modern frameworks, version control tools, microservices, DevOps practices, and cloud platforms. The course focuses on applying concepts to develop secure, scalable, and deployable software solutions.

B. Prerequisite Course -

C. Course Objectives

Learners are exposed to:

- Understand and use database queries and web technologies to build interactive applications.
- Apply Spring Boot concepts to develop backend and RESTful services.
- Use microservices architecture for building scalable applications.
- Practice version control, build tools, and DevOps workflows for software deployment.
- Explore cloud platforms and AI-assisted coding techniques for modern application development.

D. Course Outcomes

Upon the successful completion of the course, students will be able to:

COs	Course Outcomes	K Level
CO1	Use database queries and web technologies to create dynamic web pages.	**K3**
CO2	Implement backend services using Spring Boot and RESTful APIs.	**K3**
CO3	Develop microservice-based applications using Spring frameworks.	**K3**
CO4	Apply version control, build tools, and CI/CD practices for application delivery.	**K3**
CO5	Build cloud-enabled applications using cloud services and AI-based coding tools.	**K3**

E. Correlation of COs with Program Outcomes and Programme Specific Outcomes

Cos	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	3	1	2	-	-	-	-	-	-	2	3	2
CO2	3	2	3	2	3	-	-	-	-	-	-	2	3	2
CO3	3	3	3	2	3	-	-	-	-	-	-	2	3	3
CO4	2	2	2	2	3	-	-	-	-	-	-	1	3	3
CO5	2	2	3	2	3	-	-	-	-	-	-	1	3	3

3- High; 2-Medium; 1-Low

F. Course Contents

Unit I: Database Management and Web Development basics **L - 3 Hours**

SELECT statement – Aggregate functions - Sorting and Filtering Data - Joins - Subquery. Working with Date- Triggers, Indexes, and Views - Transactions, Commit, and Rollback – SQL Built-in and User-Defined Functions. HTML5 – Tags in HTML – Design HTML Forms, Handling user inputs, - use of required. CSS3. JavaScript for Dynamic Web - ES6 features - DOM manipulation – GetElementById - Form validation. Event handling - Mouse click and double click events - Keyboard events – Developer tools.

Unit II: Spring Boot Core and Backend Development **L - 3 Hours**

Spring MVC – Annotation - Spring Core: Inversion of Control and Dependency Injection – Auto wired - BeanFactory- Spring Boot Web Stack (Servlet-based Architecture) - Object-Relational Mapping Related Annotations - Data Access Layer using Spring Data JPA. RESTful API Design and Development - Exception Handling and Validation – Spring Security Implementation.

Unit III: Advanced Spring, Microservices Programming **L – 3 Hours**

Microservices Design Principles - Service Registry and Discovery - API Gateway and Load Balancing - Spring Messaging (JMS) – React JS - Web application into mobile application – Capacitor package.

Unit IV: Version Control, Build Tools and DevOps Foundations **L - 3 Hours**

Git & GitHub Fundamentals – Repository Management - Versioning - Branching, Merging. Build Automation using Maven - DevOps - Continuous Integration and Continuous Deployment (CI/CD) using Jenkins, GitHub Actions, and GitLab CI/CD.

Unit V: Cloud Computing and Vibe Coding **L - 3 Hours**

Cloud Services: API Gateway, File Storage, Compute, Relational Database Service– Pay as you Use model - Introduction to AWS - Google Cloud - Azure - Platform as a Service (PaaS) – Infrastructure as a Service (IaaS) - Software as a Service (SaaS) – Cloud applications for business. Vibe Coding - Prompt Engineering – Fine tuning the prompt - Generative AI.

TOTAL:15 Hours

G. Laboratory Experiments

Part A

Task 1: Student Registration & Data Storage

Design a Student Registration Form using HTML5 and CSS3 that collects:

Name, Email, DOB, Department, Phone

Store the data in a database table and retrieve it using SELECT.

Task 2: Data Retrieval & Sorting Dashboard

Description: Create a page that displays student or employee records with:

Sorting by name or date

Filtering by department

Count of students per department

Task 3: Login System with Validation

Description: Develop a Login Page:

Validate inputs using JavaScript
Check credentials from database
Show error messages dynamically

Real-Time Usage: Authentication systems for any web application.

Task 4: Order Management using Joins
Concepts Used: Joins, Subqueries, ORDER BY, CSS layout

Create tables for:
- Customers
- Orders
- Products

Display a customer order history using JOIN queries and a subquery to find:
- Highest value order
- Most active customer

Real-Time Usage: E-commerce and retail systems.

Task 5: Transaction-Based Payment Simulation
Description: Simulate an online payment process:
Deduct balance from user account
Add amount to merchant account
Use COMMIT on success, ROLLBACK on failure

Real-Time Usage: Banking and digital payment applications.

Task 6: Automated Logging using Triggers & Views
A trigger that logs every INSERT or UPDATE
A view that shows daily activity reports

Real-Time Usage: Audit logging in enterprise databases.

Task 7: Interactive Web Form with Events & Functions

Description: Build an interactive feedback form:
- Validate inputs on keypress
- Highlight fields on mouse hover
- Show confirmation on double-click submit
- Use JS functions for reusable validation logic

Real-Time Usage: Customer feedback and survey systems

Task 8: Create a simple Employee Management module using Spring Core. Demonstrate Inversion of Control and Dependency Injection using annotations such as @Component and @Autowired. Use BeanFactory to manage beans and store employee data in memory.

Task 9: Develop a basic Spring MVC application using annotation-based configuration. Create a controller to accept user requests and display employee details using the MVC flow without XML configuration.

Task 10: Build a Student CRUD application using Spring Boot. Map the database table using JPA annotations such as @Entity, @Id, @Table, and @Column. Perform Create, Read, Update, and Delete operations using a relational database.

Task 11: Implement a Data Access Layer using Spring Data JPA. Use JpaRepository and custom query methods to retrieve student records based on conditions such as department or age, and include sorting and pagination.

Task 12: Design and develop a RESTful API for Product Management using Spring Boot. Implement GET, POST, PUT, and DELETE methods and return JSON responses. Test the APIs using a REST client.

Task 13: Add exception handling and validation to the RESTful application. Implement global exception handling and apply validation annotations to ensure correct input data and meaningful error responses.

Task 14: Design a simple microservices-based system by splitting a monolithic application into at least two independent services. Apply microservices design principles such as loose coupling, single responsibility, and independent deployment.

Task 15: Implement Service Registry and Discovery using a tool such as Eureka. Register multiple microservices and enable dynamic service lookup instead of hard-coded service URLs.

Task 16: Configure an API Gateway to route client requests to appropriate microservices. Implement basic load balancing so that requests are distributed across multiple instances of a service.

Task 17: Enable inter-service communication using REST-based communication. Create one microservice that consumes REST APIs exposed by another microservice and handles responses and failures gracefully.

Task 18: Write unit test cases for microservices using a testing framework. Test service logic, REST controllers, and data handling independently to ensure reliability.

Task 19: Create a Git repository for a sample project and demonstrate basic Git operations such as initializing a repository, staging files, committing changes, and maintaining version history. Push the repository to GitHub and manage it using remote repositories.

Task 20: Implement branching strategies in Git by creating feature branches. Perform merging and rebasing operations, and intentionally create merge conflicts to understand and resolve them effectively.

Task 21: Implement a CI/CD pipeline using Jenkins, GitHub Actions, or GitLab CI/CD. Automate code checkout, build, test, and deployment steps, and demonstrate continuous integration and continuous deployment for the application

Task 22: Explore major cloud service providers such as AWS, Google Cloud, and Microsoft Azure. Identify and compare their core services related to compute, storage, databases, and networking, along with the pay-as-you-use pricing model.

Task 23: Apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool. Design effective prompts to generate code snippets, cloud configuration templates, or application logic, and evaluate the quality and efficiency of the generated outputs for real-world business applications.

Task 24: Use Vibe Coding to build a complete cloud-based feature using Generative AI. Write iterative prompts to generate a REST API, cloud deployment steps, and security configurations. Refine the prompts based on the output quality, document prompt versions, and evaluate how prompt engineering improves code accuracy, scalability, and real-world usability.

PART – B

Project -1 : Design and develop a React JS based web application for Ticket Booking of an Internal Department Event (such as technical fest, or seminar).

The application should allow users (students/faculty) to view event details and book tickets online.

The system must include the following requirements:
Event Details Module

Display event name
Department name
Event date and time
Venue
Ticket price
Available number of tickets
Ticket Booking Module

Allow user to enter:

Name
Email ID
Department
Number of tickets required
Prevent booking more tickets than available
Display booking confirmation message after successful booking

Validation Requirements

All input fields must be mandatory
Email ID should be in valid format
Number of tickets should be a positive number
Display appropriate error messages for invalid inputs
React Concepts to be Used
Functional components
JSX
UseState hook for state management
Event handling

Conditional rendering
Output Requirements
Display updated available ticket count after booking

Show booking summary including:

User name
Event name
Number of tickets booked
Total amount
Exception / Error Handling
Handle invalid inputs gracefully
Display user-friendly error messages
Additional Features (Optional)
Reset booking form
Simple CSS styling for better UI
Separate components for Event Details and Booking Form

The application should be developed using React JS and run in a browser.
Proper component structure and code readability must be maintained.

Full Stack Project-2: Smart Campus Event Management System

Description: A web application for a college/university to manage campus events, workshops, and seminars. Students can view upcoming events, register, and provide feedback, while admins can create, update, or delete events.

Key Features:
Student Side:

Browse upcoming events (REST APIs + MVC pages)
Register for events with form validation
View registered events

Admin Side:

Add / Edit / Delete events (CRUD using Spring Boot + JPA)
Search events using filters (date, department, type)

View event registration statistics (aggregate functions)

Technical Highlights:

- Spring Core: Dependency Injection (@Autowired), Bean management
- Spring MVC: Annotation-based controllers (@Controller, @RequestMapping)
- Spring Boot: Embedded Tomcat server, REST API development (@RestController)
- Spring Data JPA: Entity mapping (@Entity, @Id, @Table), CRUD operations
- Validation: @NotNull, @Size for form inputs
- Security: Basic authentication for admin login
- Exception Handling: Global exception handling using @ControllerAdvice
- HTML5/CSS3 + Thymeleaf for frontend

Project-3: Project Idea: Job Portal Management System

Description: A web application that connects job seekers and employers. Students can create profiles, upload resumes, browse job listings, and apply online. Employers can post jobs, view applicants, and shortlist candidates.

Key Features:

User Side (Job Seeker / Student):
Register/login and maintain profile
Upload resumes (file storage in database or local folder)
Search and filter jobs by category, location, or experience
Apply for jobs and track application status
Receive notifications for shortlisted jobs

Admin/Employer Side:
Post new jobs (title, description, skills required, salary)
Edit or delete jobs
View applicants for posted jobs
Shortlist or reject applications

Technical Highlights:
Backend (Spring Boot + Spring MVC):
REST APIs for CRUD operations
Controllers using @RestController and @Controller
Validation using @NotNull, @Size
Exception handling with @ControllerAdvice

Database (Spring Data JPA):

Entities: User, Job, Application
Relationships: One-to-Many (Employer → Jobs), Many-to-One (Application → Job/User)
Queries: Custom finder methods, sorting, and filtering

Security:
Spring Security for login and role-based access (Student, Employer, Admin)
Password hashing for security

Frontend:
HTML5/CSS3 and Thymeleaf
Responsive forms and dashboards
Dynamic table filtering (JavaScript/ES6)

Optional Add-ons (Industry-Relevant):
File uploads for resumes (using MultipartFile)
Email notifications for job status
Simple dashboard analytics (number of applications, shortlisted candidates)

Resources Needed:

- MySQL / H2 Database
- Spring Boot starter projects (can use Spring Initializr)

- Any code editor: VS Code, IntelliJ Community
- Localhost server for testing (no paid cloud required)

H. Learning Resources

i. Text Books:

1. Ashok Shah & K. K. Saxena. (2019). Web Technologies. University Press.
2. Spring in Action, Craig Walls. (2018). Fifth Edition, Manning Publications.
3. Douglas Crockford. (2008). JavaScript: The Good Parts. O'Reilly Media.
4. Jaya V. & Ramesh P. (2024). Vibe Coding and Generative AI for Developers. TechPress Publications.

ii. Reference Books:

1. Kathy Sierra & Bert Bates. (2017). Head First Java (2nd ed.). O'Reilly Media.
2. Abraham Silberschatz, Henry F. Korth, & S. Sudarshan. (2019). Database System Concepts, Seventh Edition, McGraw-Hill Education.

iii. Online Resources:

1. W3Schools India – Free tutorials for HTML, CSS, JavaScript, and more with simple examples and runnable code. Website: https://www.w3schools.in
2. LearnVern – Complete Web Design Course in Hindi covering HTML5, CSS3, and JavaScript for beginners. Website: https://www.learnvern.com/complete-web-design
3. Engineers Coding Hub – Full Stack Projects with GitHub Code including Spring Boot + HTML/CSS/JS tutorials. Website: https://engineerscodinghub.com/full-stack-projects-with-github-code.
4. Awesome Spring Boot – Curated GitHub repository with blogs, tutorials, and learning resources for Spring Boot. Website: https://github.com/RameshMF/awesome-spring-boot